SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.Harsha	**Reg. No.:**	192472325
Section / Batch:	Slot B/2024	**Date:**	

Code of Conduct

I M.Harsha (192472325) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *M.Harsha*

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

MORPHOLOGICAL ANALYSIS ERROR IN BIOMEDICAL DATASET

Aim: -To analyze the errors caused by applying Porter Stemmer to biomedical text and understand the difference between inflectional and derivational morphology.

1. Pre-processing of Biomedical Text

Biomedical text preprocessing helps remove unnecessary information and prepare medical documents for analysis. The PubMed 20k RCT dataset can be used for this experiment.

The following preprocessing steps are performed:

1. **Text Cleaning:** Remove unwanted symbols and unnecessary characters.
2. **Tokenization:** Split sentences into individual words.
3. **Lowercasing:** Convert words into lowercase.
4. **Stopword Removal:** Remove common words that do not contribute much meaning, while preserving important medical negations such as “no” and “not”.
5. **Stemming:** Apply Porter Stemmer to reduce words to their stems.

2. Application of Porter Stemmer

Porter Stemmer is a rule-based algorithm that removes certain word endings to reduce words to shorter forms. However, it does not understand the complete meaning of biomedical terms.

Examples:

Original Word	Possible Stemming Result
Studies	studi
Studying	studi
Connected	connect
Treatment	treat
Diseases	diseas

These examples illustrate how stemming can produce shortened forms that may not be valid English words or appropriate medical terms.

3. Words Where Stemming May Remove Important Information

Stemming may remove meaningful parts of biomedical words and affect their interpretation.

Examples:

- **Diagnosis and Diagnostic:** These words have related meanings but represent different grammatical forms.
- **Infection and Infectious:** These words are related but are used differently in medical contexts.
- **Inflammation and Inflammatory:** Removing suffixes may affect the distinction between a condition and its related adjective.
- **Treatment and Treating:** These words are related but have different grammatical functions.

Therefore, aggressive stemming can reduce the accuracy of biomedical information retrieval and text classification.

4. Inflectional and Derivational Morphology

Inflectional morphology changes the grammatical form of a word without changing its basic meaning or word category.

- Study → Studies
- Treat → Treated
- Patient → Patients

Derivational morphology creates a related word that may have a different meaning or grammatical category. Inflectional changes are generally safer to normalize. Derivational changes require greater care because they may carry important semantic information.

5. Result: -The analysis shows that Porter Stemmer can reduce the size of biomedical vocabularies, but it may also remove meaningful information from specialized medical terms.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

ERROR ANALYSIS OF FINITE-STATE MORPHOLOGICAL PARSER IN SOCIAL MEDIA

1. Aim: -To analyze the errors in a Finite-State Transducer (FST) used for morphological analysis of social media words and suggest improvements for handling prefixes, suffixes, and spelling changes.

2. Error Analysis

A Finite-State Transducer (FST) is an NLP technique used to identify the root words, prefixes, suffixes, and grammatical forms of words. The given social media words reveal the following errors.

Word	Correct Analysis	Error
Happiest	Happy + -est	Failure to handle y-to-i change
Unbelievable	Un- + Believe + -able	Failure to handle multiple affixes
Running	Run + -ing	Failure to handle consonant doubling
Reordering	Re- + Order + -ing	Failure to handle multiple morphological operations
Smartphones	Smart + Phone + -s	Failure to recognize compound words
Unreadable	Un- + Read + -able	Failure to handle prefix and suffix combinations

3. Causes of Errors

a. Multiple Affixes: Words such as *unbelievable* contain both a prefix and a suffix. A basic parser may fail to recognize the complete structure.

b. Spelling Changes: English words may change spelling when suffixes are added. For example, *happy* becomes *happiest*, and *run* becomes *running*.

c. Compound Words: The word *smartphones* contains the compound word *smartphone* and the plural suffix *-s*. A parser without compound-word rules may analyze it incorrectly.

d. Limited Transition Rules: A basic FST may not support all combinations of roots, prefixes, suffixes, and spelling changes.

4. Modified FST Design

The parser can be improved by adding transitions for prefix recognition, root identification, spelling changes, suffix recognition, and compound-word analysis.

The processing sequence is:

Input Word → Prefix Recognition → Root Identification → Spelling-Change Rules → Suffix Recognition → Final Output

The modified parser should recognize prefixes such as *un-* and *re-*, suffixes such as *-ing*, *-est*, and *-able*, and spelling changes such as consonant doubling and y-to-i replacement. These rules help the parser produce more accurate morphological analyses.

5. Accuracy and Complexity

Parser accuracy can be calculated using:

Accuracy = (Correct Analyses / Total Words Tested) × 100

Both the original and modified parsers should be tested on the same labeled dataset. The modified parser is expected to perform better, but the actual improvement must be verified experimentally.

For a deterministic FST, the time complexity is generally **O(N)**, where N is the length of the input word. The storage required depends on the number of states and transitions in the parser.

6. Result

The analysis identifies errors caused by multiple affixes, spelling changes, and compound words. Adding suitable morphological rules can improve the accuracy of the FST and make it more effective for processing social media text.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import
PorterStemmer import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

IDENTIFY THE ERROR, RECTIFY IT, AND ANALYZE THE OUTPUT

Aim: -To identify and correct errors in a Python program using Porter Stemmer and analyze the original and stemmed words.

1. Identification of Errors: -The original program applies ps.stem() directly to an entire news article. However, Porter Stemmer is designed to process individual words. The program must tokenize the text before stemming. It should also handle missing values and verify that the required text column exists in the dataset.

2. Explanation of Incorrect Output

Applying a stemmer directly to a complete article does not perform proper word-by-word stemming. Tokenization is necessary to separate the text into individual words. The stemmer can then remove or modify word endings to produce stemmed words.

3. Corrected Program

The corrected Python program uses Pandas to store the news text, regular expressions to tokenize the text, and NLTK's Porter Stemmer to process individual words. It displays the original text, processed text, and a comparison of 20 words.

4. Execution and Output

The program was executed in Python IDLE using sample news sentences. The output displays the original news text alongside its stemmed version. It also prints 20 original words and their corresponding stems.

```
========== RESTART: C:/Users/Reddy/Documents/NLP ASSESSMENTS/co3 q 5.py ====
ORIGINAL AND STEMMED NEWS TEXT
                                    Text                                Processed
         The players are playing football       the player are play footbal
The companies developed new technologies the compani develop new technolog
        The children were running quickly     the children were run quickli
      The organization manages businesses              the organ manag busi
      The workers are working efficiently        the worker are work effici

WORD-BY-WORD COMPARISON
the                  -> the
players              -> player
are                  -> are
playing              -> play
football             -> footbal
the                  -> the
companies            -> compani
developed            -> develop
new                  -> new
technologies         -> technolog
the                  -> the
children             -> children
were                 -> were
running              -> run
quickly              -> quickli
the                  -> the
organization         -> organ
manages              -> manag
businesses           -> busi
the                  -> the
workers              -> worker
are                  -> are
working              -> work
efficiently          -> effici
```

5. Comparison of Original and Stemmed Words

Original words retain their complete form, while stemmed words are shortened by removing or modifying endings. For example, *playing* becomes *play*, *cats* become *cat*, and *studies* becomes *studi*. Stemming reduces word variations and can help simplify text processing.

6. Analysis of 20 Stemming Cases

The 20 words displayed in the output illustrate two types of morphology:

- **Inflectional morphology:** Changes the grammatical form of a word without creating a new basic meaning. Examples include *cats* → *cat*, *played* → *play*, and *playing* → *play*.
- **Derivational morphology:** Can change a word's meaning or grammatical category. Examples include *happiness* → *happi*, *nationalization* → *nation*, and *management* → *manag*.

Result: -The Python program was corrected to tokenize text and apply Porter Stemming to individual words. The output demonstrates the comparison between original and stemmed words and identifies potential stemming errors.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
   if word.endswith("s"):
      return word[:-2], "Plural Noun"
   else:
      return word, "Singular"
words =
["cars","boxes","cities","children"] for
w in words:
   print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

ERROR ANALYSIS OF A FINITE-STATE MORPHOLOGICAL PARSER

Aim: -To identify and correct logical errors in a finite-state morphological parser and analyze different English plural noun formation rules.

1. Identification of Errors

The original parser removes the last two characters from every word ending in s. This produces incorrect results because regular plural nouns generally require only the final to be removed. The parser does not handle -ies endings, irregular plurals, or spelling changes. It also assumes that every word ending in s is a plural noun.

2. Explanation of Incorrect Output

Different English nouns form their plurals in different ways. For example, cars becomes car, boxes becomes box, and cities becomes city. Irregular forms such as children cannot be correctly analyzed by simply removing suffixes. Therefore, the original parser fails because it uses only one rule for all plural nouns.

3. Corrected Program

The corrected Python program uses a dictionary for common irregular plurals, WordNet for lexical support when available, and suffix-based rules to identify plural forms. It also handles words that do not match the known plural patterns.

```
========== RESTART:
============================================================
FINITE-STATE MORPHOLOGICAL PARSER
============================================================
WordNet available: True

Input Word      Root Word       Classification
------------------------------------------------------------
cars            car             Plural Noun
boxes           box             Plural Noun
cities          city            Plural Noun
children        child           Plural Noun
dogs            dog             Plural Noun
wishes          wish            Plural Noun
babies          baby            Plural Noun
men             man             Plural Noun
feet            foot            Plural Noun
books           book            Plural Noun

Result: Plural forms were analyzed using lexical
information and morphological suffix rules.
```

5. Analysis of Plural Formation Rules

The parser handles the following plural formation patterns:

- **Regular plurals:** The final s is removed, as in cars → car and dogs → dog.
- **Words ending in -es:** The plural ending is removed, as in boxes → box and wishes → wish.
- **Words ending in -ies:** The ending is replaced with y, as in cities → city and babies → baby.

6. Limitations of Rule-Based Finite-State Parsing

1. **Irregular plurals:** Forms such as children and mice require special rules or a lexical database.
2. **Ambiguous words:** Not every word ending in s is a plural noun.
3. **Spelling variations:** Different plural patterns require different rules.
4. **Lexicon dependency:** Unknown words may not be analyzed correctly.

Result: -The finite-state morphological parser was improved by adding rules for regular plurals, -es and -ies endings, and common irregular plurals. WordNet provides additional lexical support when available.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____